Generative AI Foundations for Engineers

Clair J. Sullivan
clair@clairsullivan.com

CLAIR SULLIVAN
& ASSOCIATES

bill

aws

Schedule

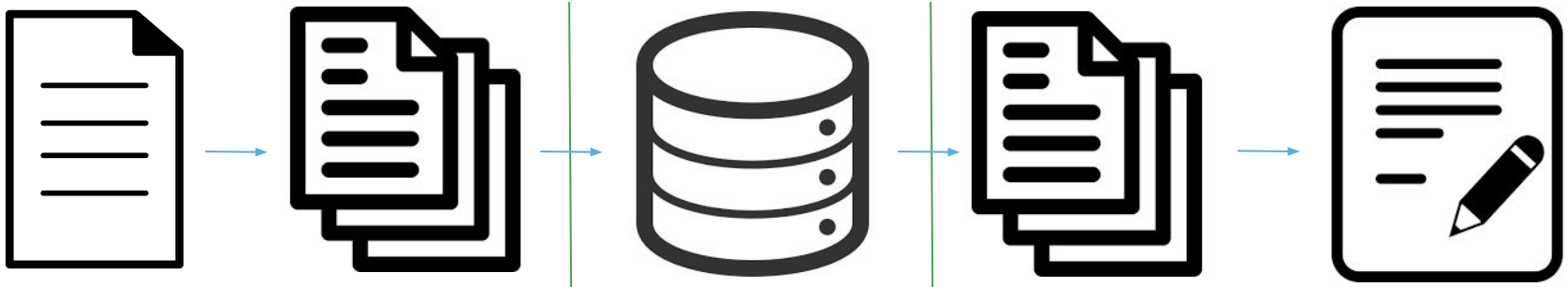
9:00 - 9:30	Check in, breakfast
9:30 - 9:45	Welcome and opening remarks
9:45 - 10:00	Overview of event
10:00 - 11:45	Module 1
12:00 - 12:30	WORKING LUNCH
12:30 - 2:00	Module 2
2:00 - 2:15	BREAK
2:15 - 3:45	Module 3
3:45 - 4:00	BREAK
4:00 - 4:20	Close competition
4:20 - 4:30	Announce winners, prizes!
4:30 - 5:30	Celebration and networking

Module 2: Retrieval-Augmented Generation (RAG) & External Data Integration

What is RAG and Why is it Important?

- LLMs are trained off of a large corpus of knowledge
  - They understand general concepts, NOT specifics
  - When they don't know the specifics, they tend to hallucinate
- RAGs providing them with additional information on relevant subjects
- Involves giving the LLM access to external data sources to improve the accuracy and relevance of generated responses
  - Documents, databases, etc.
  - Retrieves relevant context at query time
- Essential for domain-specific tasks when it is unlikely that the relevant information is in the LLM's training set

How Do We Do That?



Knowledge Base

Chunks

Database

Retrieved Chunks

Response

Text Embedding Model

Large Language Model

Splitting and Chunking

- Creating one vector of a large amount of text would dilute the meaning captured in the vector
- Need to break large amounts of text into smaller pieces
- **Chunking:** breaking a large amount of text into pieces of a fixed size, usually specified by the number of characters or `chunk_size`
- **Splitting:** creating smaller pieces of text made by breaking the full text at logical boundaries like periods and carriage returns
- Chunks should be small enough that they have distinct meaning with some overlap with neighboring chunks to allow continuity

ChunkViz v0.1

Want to learn more about AI Engineering Patterns? Join me on Twitter or Newsletter.

Language Models do better when they're focused.

One strategy is to pass a relevant subset (chunk) of your full data. There are many ways to chunk text.

This is an tool to understand different chunking/splitting strategies.

Explain like I'm 5...

The most obvious case of superlinear returns is when you're working on something that grows exponentially. For example, growing bacterial cultures. When they grow at all, they grow exponentially. But they're tricky to grow. Which means the difference in outcome between someone who's adept at it and someone who's not is very great.

Startups can also grow exponentially, and we see the same pattern there. Some manage to achieve high growth rates. Most don't. And as a result you get qualitatively different outcomes: the companies with high growth rates tend to become immensely valuable, while the ones with lower growth rates may not even survive.

Y Combinator encourages founders to focus on growth rate rather than absolute numbers. It prevents them from being discouraged early

Upload .txt

Splitter: Character Splitter 🦜🔗

Chunk Size: 25

Chunk Overlap: 0

Total Characters: 2658
Number of chunks: 107
Average chunk size: 24.8

One of the most important things I didn't understand about the world when I was a child is the degree to which the returns for performance are superlinear.

Teachers and coaches implicitly told us the returns were linear. "You get out," I heard a thousand times, "what you put in." They meant well, but this is rarely true. If your product is only half as good as your competitor's, you don't get half as many customers. You get no customers, and you go out of business.

It's obviously true that the returns for performance are superlinear in business. Some think this is a flaw of capitalism, and that if we changed the rules it would stop being true. But superlinear returns for performance are a feature of the world, not an artifact of rules we've invented. We see the same pattern in fame, power, military victories, knowledge, and even benefit to humanity. In all of these, the rich get richer. [1]

You can't understand the world without understanding the concept of superlinear returns. And if you're ambitious you definitely should, because this will be the wave you surf on.

It may seem as if there are a lot of different situations with superlinear returns, but as far as I can tell they reduce to two fundamental causes: exponential growth and thresholds.

The most obvious case of superlinear returns is when you're working on something that grows exponentially. For example, growing bacterial cultures. When they grow at all, they grow exponentially. But they're tricky to grow. Which means the difference in outcome between someone who's adept at it and someone who's not is very great.

Startups can also grow exponentially, and we see the same pattern there. Some manage to achieve high growth rates. Most don't. And as a result you get qualitatively different outcomes: the companies with high growth rates tend to become immensely valuable, while the ones with lower growth rates may not even survive.

Y Combinator encourages founders to focus on growth rate rather than absolute numbers. It prevents them from being discouraged early on, when the absolute numbers are still low. It also helps them decide what to focus on: you can use growth rate as a compass to tell you how to evolve the company. But the main advantage is that by focusing on growth rate you tend to get something that grows exponentially.

YC doesn't explicitly tell founders that with growth rate "you get out what you put in," but it's not far from the truth. And if growth rate were proportional to performance, then the reward for performance p over time t would be proportional to pt.

ChunkViz v0.1

Want to learn more about AI Engineering Patterns? Join me on Twitter or Newsletter.

Language Models do better when they're focused.

One strategy is to pass a relevant subset (chunk) of your full data. There are many ways to chunk text.

This is an tool to understand different chunking/splitting strategies.

Explain like I'm 5...

```
The most obvious case of superlinear returns is when you're working on something that grows exponentially. For example, growing
bacterial cultures. When they grow at all, they grow exponentially. But they're tricky to grow. Which means the difference in
outcome between someone who's adept at it and someone who's not is very great.

Startups can also grow exponentially, and we see the same pattern there. Some manage to achieve high growth rates. Most don't. And
as a result you get qualitatively different outcomes: the companies with high growth rates tend to become immensely valuable, while
the ones with lower growth rates may not even survive.

Y Combinator encourages founders to focus on growth rate rather than absolute numbers. It prevents them from being discouraged early
```

Upload .txt

Splitter: Recursive Character Text Splitter 🦜🔗

Chunk Size: 481

Chunk Overlap: 0

Total Characters: 2658
Number of chunks: 7
Average chunk size: 379.7

One of the most important things I didn't understand about the world when I was a child is the degree to which the returns for performance are superlinear.

Teachers and coaches implicitly told us the returns were linear. "You get out," I heard a thousand times, "what you put in." They meant well, but this is rarely true. If your product is only half as good as your competitor's, you don't get half as many customers. You get no customers, and you go out of business.

It's obviously true that the returns for performance are superlinear in business. Some think this is a flaw of capitalism, and that if we changed the rules it would stop being true. But superlinear returns for performance are a feature of the world, not an artifact of rules we've invented. We see the same pattern in fame, power, military victories, knowledge, and even benefit to humanity. In all of these, the rich get richer. [1]

You can't understand the world without understanding the concept of superlinear returns. And if you're ambitious you definitely should, because this will be the wave you surf on.

It may seem as if there are a lot of different situations with superlinear returns, but as far as I can tell they reduce to two fundamental causes: exponential growth and thresholds.

The most obvious case of superlinear returns is when you're working on something that grows exponentially. For example, growing bacterial cultures. When they grow at all, they grow exponentially. But they're tricky to grow. Which means the difference in outcome between someone who's adept at it and someone who's not is very great.

Startups can also grow exponentially, and we see the same pattern there. Some manage to achieve high growth rates. Most don't. And as a result you get qualitatively different outcomes: the companies with high growth rates tend to become immensely valuable, while the ones with lower growth rates may not even survive.

Y Combinator encourages founders to focus on growth rate rather than absolute numbers. It prevents them from being discouraged early on, when the absolute numbers are still low. It also helps them decide what to focus on: you can use growth rate as a compass to tell you how to evolve the company. But the main advantage is that by focusing on growth rate you tend to get something that grows exponentially.

YC doesn't explicitly tell founders that with growth rate "you get out what you put in," but it's not far from the truth. And if growth rate were proportional to performance, then the reward for performance p over time t would be proportional to pt.

What are the Embeddings?

- All GenAI is just math
- Need to convert text into numbers so that the LLM can do math on them
  - Predictions of next most likely word given previous words
- An n-dimensional **vector** (a list of floats) with each value between -1.0 and 1.0
  - Which model you use determines n, but numbers > 1000 are typical
- Stored in a database for later retrieval based on **similarity**
  - Cosine similarity is easy to do on vectors, which allows the LLM to quickly find the most relevant pieces of information in your database
- Remember: everything is about converting strings (or other data types) to vectors, meaning you need to start with strings!

Schedule

9:00 - 9:30	Check in, breakfast
9:30 - 9:45	Welcome and opening remarks
9:45 - 10:00	Overview of event
10:00 - 11:45	Module 1
12:00 - 12:30	WORKING LUNCH
12:30 - 2:00	Module 2
2:00 - 2:15	BREAK
2:15 - 3:45	Module 3
3:45 - 4:00	BREAK
4:00 - 4:20	Close competition
4:20 - 4:30	Announce winners, prizes!
4:30 - 5:30	Celebration and networking